

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x08 实验报告

VGA 显示输出

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. 实现方案

本实验的目标是实现 VGA 显示输出，为用户提供 GUI，核心流程可以概括为以下几步：

1. 在 bootloader 中，通过 `get_handle_for_protocol::<GraphicsOutput>()` 获取 GOP 信息（帧缓冲地址、分辨率、像素格式等），并传递给内核
2. 在内核中，基于 GOP 信息初始化 VGA 帧缓冲驱动，支持逐像素绘制、矩形填充、清屏、滚动等操作
3. 利用 `embedded-graphics` 库实现字符渲染：将内置的 8*16 位图字体转换为 `MonoFont`，通过 `DirectFb`（栈上 `DrawTarget`，绕开 VGA 互斥锁直接写入帧缓冲，因为我们的 VGA 互斥锁会被 `clock_demo` 线程获取用于绘制时钟，如果要进行日志输出需要绕开）进行绘制
4. 实现一个 VGA 文本控制台，支持字符输出、换行、滚屏、回滚缓冲区等功能，并将其对接到日志系统
5. 实现分屏布局与动画模拟时钟：上半部分使用 Bresenham 直线算法 + 方形笔刷绘制粗线指针和刻度，借助 `embedded-graphics::Text` 渲染小时数字，基于 UEFI 时间进行帧同步；下半部分为文本控制台
6. 激活键盘中断，通过与串口共享输入缓冲区的方式响应键盘输入

2. 在 Bootloader 中获取图形输出信息

2.1. UEFI Graphics Output Protocol（GOP）

UEFI 固件提供了一套标准的协议（Protocol）供 bootloader 使用，其中之一就是 Graphics Output Protocol（GOP）。GOP 可以为 bootloader 提供一个线性的帧缓冲（Linear Framebuffer），使得我们可以直接通过内存写入的方式控制屏幕上的每一个像素。

GOP 协议提供了以下关键信息：

- `frame_buffer()`：帧缓冲的起始物理地址和总大小（以字节为单位）
- `current_mode_info()`：当前显示模式的信息，包括：
  - `resolution()`：屏幕分辨率，返回 `(width, height)` 元组，单位为像素
  - `stride()`：每一行像素占用的像素数（可能大于 `width * bytes_per_pixel`，因为存在对齐填充）
  - `pixel_format()`：像素格式，常见值为 `Rgb`（红绿蓝）、`Bgr`（蓝绿红）、`Bitmask` 等

2.2. `get_gop_info()` 函数的实现

我们在 bootloader（`crates/boot/src/main.rs`）中添加了 `get_gop_info()` 函数，其实现如下：

```rust
/// Query UEFI Graphics Output Protocol for framebuffer information
fn get_gop_info() -> Option<crate::GopInfo> {
    let handle = match uefi::boot::get_handle_for_protocol::<GraphicsOutput>() {
        Ok(h) => h,
        Err(e) => {
            warn!("Failed to get GraphicsOutput handle: {:?}", e);
            return None;
        }
    };
    let mut gop = match
uefi::boot::open_protocol_exclusive::<GraphicsOutput>(handle) {
        Ok(g) => g,
        Err(e) => {
            warn!("Failed to open GraphicsOutput protocol: {:?}", e);
            return None;
        }
    };
    let mode_info = gop.current_mode_info();
    let fb = gop.frame_buffer();

    let fb_addr = gop.frame_buffer().as_mut_ptr() as u64;

    // Determine pixel format
    let format = match mode_info.pixel_format() {
        PixelFormat::Rgb => 0,
        PixelFormat::Bgr => 1,
        PixelFormat::Bitmask => 2,
        PixelFormat::BltOnly => 3,
    };

    let info = crate::GopInfo {
        fb_addr,
        fb_size: gop.frame_buffer().size() as u64,
        width: mode_info.resolution().0 as u32,
        height: mode_info.resolution().1 as u32,
        stride: mode_info.stride() as u32,
        format,
    };

    info!(
        "GOP Info: {}x{}, stride={}, fb=0x{:x}, fmt={}",
        info.width, info.height, info.stride, info.fb_addr, info.format
    );

    Some(info)
}
```

具体而言，有以下五个步骤:

1. **获取协议句柄**：调用 `uefi::boot::get_handle_for_protocol::<GraphicsOutput>()` 查找系统中支持 GOP 的硬件句柄。该函数返回 `Result<Handle, Error>`，失败则说明当前平台不支持图形输出（例如纯串口环境），此时返回 `None`。
2. **打开协议**：通过 `uefi::boot::open_protocol_exclusive::<GraphicsOutput>(handle)` 以独占方式打开 GOP 协议。这会返回一个 GOP 对象的互斥引用，保证在 UEFI 环境下不会与其他代码产生竞争。打开协议意味着我们获得了对显卡帧缓冲的控制权。
3. **读取模式信息**：`gop.current_mode_info()` 返回当前显示模式的元信息。我们从中提取分辨率（`resolution()`）、像素格式（`pixel_format()`）和每行步长（`stride()`）。
4. **读取帧缓冲地址**：`gop.frame_buffer()` 返回帧缓冲的物理地址和大小。我们通过 `.as_mut_ptr() as u64` 将指针转换为物理地址数字，供内核后续进行物理地址到虚拟地址的映射。
5. **封装为 `GopInfo` 结构体**：将所有信息填入 `GopInfo` 结构体中返回。

2.3. `GopInfo` 与 `BootInfo` 结构体

我们在 `crates/boot/src/lib.rs` 中定义了 `GopInfo` 结构体，用于在 bootloader 与内核之间传递图形信息：

```rust
/// Graphics Output Protocol info passed from bootloader to kernel
#[derive(Debug, Clone, Copy)]
pub struct GopInfo {
    /// Physical address of the framebuffer
    pub fb_addr: u64,
    /// Framebuffer size in bytes
    pub fb_size: u64,
    /// Horizontal resolution in pixels
    pub width: u32,
    /// Vertical resolution in pixels
    pub height: u32,
    /// Pixels per scanline
    pub stride: u32,
    /// Pixel format
    pub format: u32,
}
```

该结构体被嵌入到 `BootInfo` 结构体中，作为可选字段 `gop_info: Option<GopInfo>`，与其他启动信息（内存映射、内核页表、加载的应用程序等）一同传递给内核：

```rust
pub struct BootInfo<'a> {
    pub memory_map: BootMemoryMap,
    pub physical_memory_offset: u64,
    pub system_table: NonNull<core::ffi::c_void>,
    pub log_level: &'a str,
    pub loaded_apps: Option<AppList>,
    pub kernel_pages: KernelPages,
    // graphics info from GOP
    pub gop_info: Option<GopInfo>,
}
```

在 `efi_main()` 中，我们在退出 UEFI Boot Services **之前**调用 `get_gop_info()`。这是因为 GOP 协议属于 Boot Services，一旦调用 `uefi::boot::exit_boot_services()` 之后，所有 Boot Services 协议都将不可用。

具体实现如下，我们把得到的 GOP 信息放入 `bootinfo` 中，然后随着内核启动时需要的其他信息一起传给内核，

```rust
// 6.5 Query GOP info before exiting boot services
let gop_info = get_gop_info();

// 7. Exit boot and jump to ELF entry
info!("Exiting boot services...");

let mmap = unsafe { uefi::boot::exit_boot_services(None) };

// construct BootInfo
let bootinfo = BootInfo {
    memory_map: mmap.entries().copied().collect(),
    physical_memory_offset: config.physical_memory_offset,
    system_table,
    log_level: config.log_level,
    loaded_apps: apps,
    kernel_pages,
    gop_info,        // Get GOP for kernel
};

// align stack to 8 bytes
let stacktop = stack_start + stack_size * 0x1000 - 8;

jump_to_entry(&bootinfo, stacktop);
```

3. VGA 帧缓冲驱动的实现

3.1. `VgaDriver` 结构体

VGA 驱动定义在 `crates/kernel/src/drivers/vga.rs` 中，我们使用原始指针 `*mut u8` 访问帧缓冲内存，这样设计的原因是，我们后续将进行锁的设计，绘制时钟的进程获取锁重新绘制时钟，为了不引起日志输出等的卡顿，这部分使用裸指针访问绕开锁机制。这里的 `cursor_x` 和 `cursor_y` 参数表示当前输入光标所在的位置，为了方式多线程写导致混乱，采用了 `AtomicU32` 原子类型。

```rust
pub struct VgaDriver {
    /// Virtual address of the framebuffer (byte-level access)
    fb: *mut u8,
    /// Framebuffer size in bytes
    fb_size: u64,
    /// Horizontal resolution
    width: u32,
    /// Vertical resolution
    height: u32,
    /// Bytes per scanline
    stride_bytes: u32,
    /// Cursor X position (in character cells)
    cursor_x: AtomicU32,
    /// Cursor Y position (in character cells)
    cursor_y: AtomicU32,
}
```

3.2. 初始化和构造函数

然后我们将使用 `bootinfo` 传入内核的 `gop_info` 信息传入 `VgaDriver` 中，由于可能会因为内存对齐而导致每行的像素个数不是 `width * byte_per_scanline`，因此我们需要使用之前记录好的每行像素数来计算得到，从而让我们可以正确根据坐标定义得到某个输出位置对应字节的地址。

```
impl VgaDriver {
    pub unsafe fn new(gop_info: &GopInfo) -> Self {
        let fb_virt = physical_to_virtual(gop_info.fb_addr) as *mut u8;
        let stride_bytes = gop_info.stride * 4;

        info!(
            "VGA: fb_virt=0x{:x}, w={}, h={}, stride_bytes={}, fb_size={}",
            fb_virt as u64, gop_info.width, gop_info.height, stride_bytes,
gop_info.fb_size
        );

        Self {
            fb: fb_virt,
            fb_size: gop_info.fb_size,
            width: gop_info.width,
            height: gop_info.height,
            stride_bytes,
            cursor_x: AtomicU32::new(0),
            cursor_y: AtomicU32::new(0),
        }
    }
}
```

关键点：

- `gop_info.stride * 4`：GOP 的 stride 单位是像素，每个像素占 4 个 bytes（因为 QEMU 默认的 VGA caiyong1 BGRx 格式），因此字节步长为 `stride * 4`
- `fb_virt` 是帧缓冲在虚拟地址空间中的起始地址，后续所有像素写入操作都通过此指针进行

3.3. 像素格式

QEMU 默认的 VGA 帧缓冲使用 BGRx 格式（即每个像素 4 字节，从低到高依次为 Blue、Green、Red、未使用字节）。我们定义了 `Color` 类来表示像素颜色，同时我们也加入了一些颜色常量，便于绘图，同时我们为 `Color` 实现了 embedded-graphics 需要的 PixelColor trait。

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Color(u32);

impl Color {
    pub const BLACK: Self = Self(0x00000000);
    pub const WHITE: Self = Self(0x00FFFFFF);
    pub const RED: Self = Self(0x00FF0000);
    pub const GREEN: Self = Self(0x0000FF00);
    pub const BLUE: Self = Self(0x000000FF);
    // ...

    pub const fn new(r: u8, g: u8, b: u8) -> Self {
        Self(0x00000000 | ((r as u32) << 16) | ((g as u32) << 8) | (b as u32))
    }
}

impl PixelColor for Color {
    type Raw = RawU32;
}
```

颜色在 u32 中的布局为 `0x00RRGGBB`，虽然帧缓冲实际的字节顺序是 BGRx（即 B 在低字节，G 在中低字节，R 在中高字节），但由于我们在逐字节写入时手动编排 B、G、R 的顺序，所以 `Color` 内部使用逻辑上的 RGB 顺序存储。

3.4. 基本绘制操作

驱动提供了以下几个基本的像素级操作：

`draw_pixel`：逐像素绘制，即根据给出的 `x`， `y` 和 `color` 参数绘制该点的颜色，

```rust
pub fn draw_pixel(&self, x: u32, y: u32, color: Color) {
    if x < self.width && y < self.height {
        unsafe {
            let base = (y as usize) * (self.stride_bytes as usize) + (x as usize)
* 4;
            let c = color.0;
            self.fb.add(base).write_volatile((c & 0xFF) as u8);          // B
            self.fb.add(base + 1).write_volatile(((c >> 8) & 0xFF) as u8);  // G
            self.fb.add(base + 2).write_volatile(((c >> 16) & 0xFF) as u8); // R
        }
    }
}
```

这里使用 `write_volatile` 而非普通写入的原因是，帧缓冲属于 MMIO 区域，编译器不能由于代码只是对其写入而没有读入而对其进行优化，因为其会对 VGA 显示输出造成影响。

`fill_rect`：矩形填充，即我们在 `[x, x + w -1] * [y, y + h - 1]` 的区域内绘制同一个颜色 `color`，

```rust
pub fn fill_rect(&self, x: u32, y: u32, w: u32, h: u32, color: Color) {
    let x_end = (x + w).min(self.width);
    let y_end = (y + h).min(self.height);
    let c = color.0;
    let b = (c & 0xFF) as u8;
    let g = ((c >> 8) & 0xFF) as u8;
    let r = ((c >> 16) & 0xFF) as u8;

    for py in y..y_end {
        unsafe {
            let row_base = (py as usize) * (self.stride_bytes as usize);
            for px in x..x_end {
                let base = row_base + (px as usize) * 4;
                self.fb.add(base).write_volatile(b);
                self.fb.add(base + 1).write_volatile(g);
                self.fb.add(base + 2).write_volatile(r);
            }
        }
    }
}
```

`clear_screen`：清屏，即对整个屏幕以指定颜色 `color` 填充，同时将光标重置到 `(0, 0)`，这部分代码比较简单，这里不再赘述。

`scroll_up`：向上滚动若干字符行 `lines`，每个字符的高度为 `char_height`，需要将指定像素高度的区域整体向上复制，底部以背景色 `bg_color` 填充。用于实现文本终端的滚屏效果，可以让我们的 GUI 在屏幕上半部分有个固定的时钟的情况下可以看到历史记录，

```rust
pub fn scroll_up(&self, lines: u32, char_height: u32, bg_color: Color) {
    let pixel_lines = lines * char_height;
    if pixel_lines >= self.height {
        self.clear_screen(bg_color);
        return;
    }
    let row_bytes = (self.width as usize) * 4;

    for y in 0..(self.height - pixel_lines) {
        let src_y = y + pixel_lines;
        unsafe {
            let src = self.fb.add((src_y as usize) * (self.stride_bytes as
usize));
            let dst = self.fb.add((y as usize) * (self.stride_bytes as usize));
            core::ptr::copy_nonoverlapping(src, dst, row_bytes);
        }
    }
    let clear_start_y = self.height - pixel_lines;
    self.fill_rect(0, clear_start_y, self.width, pixel_lines, bg_color);
}
```

3.5. `embedded-graphics` 集成

为了使 `VgaDriver` 能够与 `embedded-graphics` 兼容，我们实现了 `VgaDisplay` 包装器以及所需的 traits，

```rust
pub struct VgaDisplay<'a> {
    vga: &'a VgaDriver,
}

impl OriginDimensions for VgaDisplay<'_> {
    fn size(&self) -> Size {
        Size::new(self.vga.width, self.vga.height)
    }
}

impl DrawTarget for VgaDisplay<'_> {
    type Color = Color;
    type Error = core::convert::Infallible;

    fn draw_iter<I>(&mut self, pixels: I) -> Result<(), Self::Error>
    where
        I: IntoIterator<Item = Pixel<Self::Color>>,
    {
        for Pixel(Point { x, y }, color) in pixels {
            if x >= 0 && y >= 0 {
                self.vga.draw_pixel(x as u32, y as u32, color);
            }
        }
        Ok(())
    }

    // ... See vga.rs for whole code
}
```

通过实现 `OriginDimensions` 和 `DrawTarget` trait，我们可以使用 `embedded-graphics` 提供的各种方法，将绘制的结果直接渲染到 VGA 帧缓冲上。

3.6. 全局实例与互斥保护

由于 VGA 帧缓冲是多线程共享的硬件资源，我们使用自旋锁 `spin::Mutex` 保护对驱动的访问：

```rust
once_mutex!(pub VGA: VgaDriver);
guard_access_fn!(pub get_vga(VGA: VgaDriver));

pub fn init(gop_info: &GopInfo) {
    let driver = unsafe { VgaDriver::new(gop_info) };
    init_VGA(driver);
    get_vga_for_sure().clear_screen(Color::BLACK);
    info!(
        "VGA Driver Initialized: {}x{}",
        gop_info.width, gop_info.height
    );
}
```

`once_mutex!` 宏创建一个 `spin::Once<spin::Mutex<T>>` 全局变量，确保只初始化一次。`guard_access_fn!` 宏生成 `get_vga()` 非阻塞加锁和 `get_vga_for_sure()` 阻塞式加锁两个访问函数。

4. 利用 embedded-graphics 实现字符渲染

embedded-graphics 库要求我们实现三个 traits，即 `PixelColor`，`OriginDimensions` 以及 `DrawTarget`，这三个 traits 我们在前面已经实现了，因此我们下面可以直接使用其库内的方法了！

4.1. 字体定义

为实现字符渲染，我们需要一个等宽位图字体，本实验使用了一个经典的 8×16 像素的 ASCII 字体，定义在 `crates/kernel/src/drivers/font8x16.rs` 中。

字体覆盖 ASCII 可打印字符 0x20（空格）到 0x7E（波浪号），共 95 个字符。每个字符的字形数据为 16 字节（16 行，每行 8 位），编码形式为一个 `[[u8; 16]; 95]` 数组。如字符 `'A'`（ASCII 0x41，数组索引 `0x41 - 0x20 = 33`）的字形数据为：

```rust
[0x00, 0x00, 0x10, 0x38, 0x6C, 0xC6, 0xC6, 0xFE,
 0xC6, 0xC6, 0xC6, 0xC6, 0x00, 0x00, 0x00, 0x00],
```

每一位为 1 的位置表示该像素应以前景色填充，为 0 则以背景色填充。

4.2. VGA 文本控制台 `VgaConsole`

`VgaConsole`（定义在 `crates/kernel/src/drivers/vga_text.rs`）是 VGA 文本输出的核心组件。它保存了帧缓冲的引用，支持字符绘制、光标管理、行自动换行、滚屏等功能。这里使用的是裸指针写入，为了绕开前面我们对 `VgaDriver` 实现的锁机制。

定义如下，

```rust
pub struct VgaConsole {
    char_width: u32,        // 字符宽度（8 像素）
    char_height: u32,       // 字符高度（16 像素）
    cols: u32,              // 每行字符数
    rows: u32,              // 行数
    fg_color: Color,        // 前景色
    bg_color: Color,        // 背景色
    cursor_x: u32,          // 光标横坐标
    cursor_y: u32,          // 光标纵坐标
    y_offset: u32,          // 垂直像素偏移
    fb: *mut u8,            // 直接帧缓冲指针
    fb_width: u32,          // 帧缓冲宽度
    fb_height: u32,         // 帧缓冲高度
    fb_stride_bytes: u32,   // 每行字节数
    // 滚动缓冲区
    scrollback: VecDeque<String>,
    scroll_pos: u32,
    current_line: String,
    // ... See vga_text.rs for whole code
}
```

字符绘制：`draw_char_at`

```rust
fn draw_char_at(&self, ch: char, col: u32, row: u32, fg: Color, bg: Color) {
    let px = (col * self.char_width) as i32;
    let py = (self.y_offset + row * self.char_height) as i32;

    let mut fb = DirectFb {
        fb: self.fb,
        fb_stride_bytes: self.fb_stride_bytes,
        fb_width: self.fb_width,
        fb_height: self.fb_height,
    };

    let style = MonoTextStyleBuilder::new()
        .font(get_font_8x16())
        .text_color(fg)
        .background_color(bg)
        .build();

    let mut buf = [0u8; 4];
    let s = ch.encode_utf8(&mut buf);
    let _ = Text::with_text_style(
        s,
        Point::new(px, py),
        style,
        TextStyleBuilder::new().baseline(Baseline::Top).build(),
    )
    .draw(&mut fb);
}
```

我们在这里实现了一个 `DirectFb` 类，它直接通过 `VgaConsole` 保存的帧缓冲裸指针写入像素，不经过 `VgaDriver` 的互斥锁。这保留了原有代码的死锁避免策略：日志系统在中断上下文中调用 `try_write_to_vga` 时，即使 VGA 锁被时钟绘制线程持有，文本输出仍能正常进行。

`MonoTextStyleBuilder` 将前景色和背景色绑定到 `MonoFont` 上，`Text::with_text_style(...).draw(&mut fb)` 在渲染时自动将字形的每一位映射为前景/背景 `Color` 值，并对每个字符单元格的矩形区域进行填充，将“填充背景”和“绘制前景”两步合为一次 `fill_contiguous` 调用，一定程度上提升了性能。

`write_char`：单字符写入，

处理 `\n`（换行）、`\r`（回车）、`\x08`（退格）、`\t`（制表符）以及可打印字符。当光标超出右边界时自动换行，超出底部时自动向上滚动。这部分代码请见 `vga_text.rs`。

然后实现 `write_str` 和 `fmt::Write`，实现了 `fmt::Write` trait 后，便可以使用 Rust 的 `write!` 和 `writeln!` 宏直接向 VGA 控制台输出格式化文本。

```rust
pub fn write_str(&mut self, s: &str) {
    for ch in s.chars() {
        self.write_char(ch);
    }
}

impl fmt::Write for VgaConsole {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        self.write_str(s);
        Ok(())
    }
}
```

4.3. 滚动回看 Scrollback 机制

控制台维护了一个最多 1000 行的 `VecDeque<String>` 滚动缓冲区。所有被滚动出屏幕的文本行都会存入该缓冲区。

当用户按上/下方向键时，即由键盘中断驱动调用 `scroll_console_up()` / `scroll_console_down()` 时，控制台会重新渲染滚动缓冲区中的历史内容，重新渲染使用的如下实现的的 `render_body` 函数，`scrollback` 保存了滚动的缓存数据，通过自身记录的 `scroll_pos` 和缓冲区大小计算得出当前屏幕上应该显示的部分，然后调用 `render_line_at` 函数逐行绘制，

```rust
pub fn render_body(&mut self) {
    let body_rows = self.rows - HEADER_ROWS;
    let n = self.scrollback.len() as u32;
    let bottom_idx = n.saturating_sub(self.scroll_pos);
    let top_idx = bottom_idx.saturating_sub(body_rows);

    self.clear_body();
    for i in top_idx..bottom_idx {
        let screen_row = HEADER_ROWS + (i - top_idx);
        if let Some(line) = self.scrollback.get(i as usize) {
            self.render_line_at(line, screen_row);
        }
    }
}
```

当 `scroll_pos == 0` 时，控制台处于“实时模式”的新输出直接绘制到屏幕上。当 `scroll_pos > 0` 时，新输出被缓冲到 `scrollback` 而不触及帧缓冲，并设置 `dirty_while_scrolled` 标志以提示用户有新的输出。

5. Shell 输出与日志系统对接

5.1. 日志的 VGA 输出

为了实现日志在 VGA 屏幕上的同步显示，我们修改了日志系统。在 `crates/kernel/src/utils/logger.rs` 中:

```
fn log(&self, record: &Record) {
    if self.enabled(record.metadata()) {
        // Always output to serial
        crate::utils::print_internal_no_vga(format_args!(
            "[{}] {} - {}\n\r",
            record.level(),
            record.target(),
            record.args()
        ));
        // Best-effort output to VGA (try_lock — safe from IRQ context)
        crate::drivers::vga_text::try_write_to_vga(format_args!(
            "[{}] {} - {}\n",
            record.level(),
            record.target(),
            record.args()
        ));
    }
}
```

这里的 VGA console 使用 `try_write_to_vga` 非阻塞写入，避免因为在中断调用的时候，由于原进程没有释放锁的情况输出日志下造成死锁。

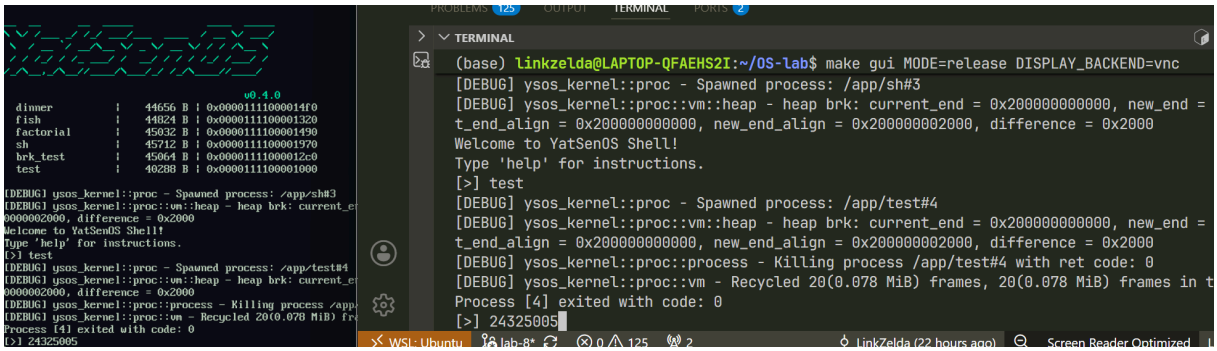
5.2. `print!` / `println!` 的双通道输出

在 `crates/kernel/src/utils/macros.rs` 中，`print_internal` 函数同时向串口和 VGA 控制台输出:

```
#[doc(hidden)]
pub fn print_internal(args: Arguments) {
    interrupts::without_interrupts(|| {
        if let Some(mut serial) = get_serial() {
            let _ = serial.write_fmt(args);
        }
        // Spin-lock VGA console (if initialized) to never drop output
        if let Some(mut console) =
crate::drivers::vga_text::get_vga_console_spin() {
            use core::fmt::Write;
            let _ = console.write_fmt(args);
        }
    });
}
```

`get_vga_console_spin()` 通过自旋等待获取锁，因为我们绘制时钟进程使用的是 `VgaDriver` 的锁，而且键盘中断不会自旋等待，且此处关了中断，因此这里自旋等待锁不会造成死锁。

我们的 shell 通过系统调用进行输入输出。shell 的 `stdin` 读取会最终调用 `input::get_line()`，后者阻塞地从共享输入缓冲区（`INPUT_BUF`）读取字符，读取到的字符同时回显到串口和 VGA 控制台，最终效果如下，可以看到我们的命令行和 GUI 均得到了同样的输出，



6. 加分项

6.1. 键盘输入实现 IO 交互

6.1.1. PS/2 键盘控制器

PS/2 键盘控制器使用 I/O 端口 `0x60`（数据端口）和 `0x64`（状态/命令端口）与 CPU 通信。驱动实现在 `crates/kernel/src/drivers/keyboard.rs` 中。

关键操作：

- 读取数据前：检查状态端口 `bit 0`（OUTPUT_FULL），为 1 表示有数据待读取
- 写入数据前：检查状态端口 `bit 1`（INPUT_FULL），为 0 表示控制器可接收命令

驱动使用 `pc-keyboard` crate 将原始扫描码（Scancode Set 1）解码为 Unicode 字符或 RawKey 事件。

6.1.2. 中断注册

键盘中断为 IRQ1，通过 IO APIC 路由到 CPU0。在 `crates/kernel/src/interrupt/keyboard.rs` 中：

```rust
pub unsafe fn register_idt(idt: &mut InterruptDescriptorTable) {
    idt[Interrupts::IrqBase as u8 + Irq::Keyboard as
u8].set_handler_fn(keyboard_handler);
}

pub extern "x86-interrupt" fn keyboard_handler(_st: InterruptStackFrame) {
    let scancode: u8 = unsafe
{ x86_64::instructions::port::Port::new(0x60).read() };
    let mut kb = get_ps2_keyboard_for_sure();
    kb.handle_scancode(scancode);
    super::ack(); // 向 LAPIC 发送 EOI
}
```

在 `interrupt/mod.rs` 中激活键盘 IRQ：

```rust
enable_irq(Irq::Keyboard as u8, 0); // enable IRQ1 for CPU0
```

6.1.3. 共享输入缓冲区

键盘输入和串口输入共用同一个无锁队列 `INPUT_BUF`（定义在 `crates/kernel/src/drivers/input.rs` 中）：

```rust
type Key = u8;

lazy_static! {
    static ref INPUT_BUF: ArrayQueue<Key> = ArrayQueue::new(128);
}

#[inline]
pub fn push_key(key: Key) {
    let _ = INPUT_BUF.push(key);
}

#[inline]
pub fn try_pop_key() -> Option<Key> {
    INPUT_BUF.pop()
}
```

`ArrayQueue` 来自 `crossbeam_queue` crate，是一个无锁的 MPSC 多生产者单消费者队列。两个中断处理程序，即键盘和串口都可以安全地向队列推送数据。

6.1.4. `get_line()` 的行编辑功能

`input::get_line()` 加入了在写入串口的同时向 VGA 控制台回显字符的功能，

```rust
pub fn get_line() -> String {
    let mut serial_port = SerialPort::new(0x3F8);
    let mut line = String::with_capacity(128);
    loop {
        let ch = pop_key();
        match ch {
            b'\n' | b'\r' => {
                write!(serial_port, "\n").unwrap();
                echo_to_vga('\n');
                break;
            }
            0x08 | 0x7F => {
                if !line.is_empty() {
                    line.pop();
                    write!(serial_port, "\x08 \x08").unwrap();
                    echo_to_vga('\x08');
                }
            }
            _ => {
                if ch >= 0x20 && ch <= 0x7E {
                    line.push(ch as char);
                    write!(serial_port, "{}", ch as char).unwrap();
                    echo_to_vga(ch as char);
                }
            }
        }
    }
    line
}
```

6.1.5. 方向键的滚动控制

在键盘驱动的 `handle_scancode` 方法中，我们捕获了上/下方向键和 PageUp/PageDown 键被映射为控制台滚动操作，其他可见字符放入输入缓冲区，而这些不可见字符在这里提前处理，这使得用户可以在 QEMU GUI 窗口中使用方向键翻阅控制台的历史输出。

```rust
// See keyboard.rs for all code
DecodedKey::RawKey(key) => {
    match key {
        KeyCode::ArrowUp => vga_text::scroll_console_up(),
        KeyCode::ArrowDown => vga_text::scroll_console_down(),
        KeyCode::PageUp => vga_text::scroll_console_up_pages(1),
        KeyCode::PageDown => vga_text::scroll_console_down_pages(1),
    }
}
```

这里的 `scroll_console_...` 方法，内部采用 `try_lock` 获得锁，因此不会造成在某个进程进行键盘输入中断时发生死锁的情况。

通过上述键盘中断驱动与共享输入缓冲区的实现，我们成功实现了 QEMU GUI 窗口中的输入输出交互。用户可以直接在 QEMU GUI 窗口中：

- 看到彩色文本的 Shell 提示符
- 键入命令并看到实时回显
- 使用方向键和 PageUp/PageDown 翻看历史输出
- 所有输出同时出现在串口和 VGA 屏幕上

6.2. 分屏与动画时钟

我们实现了分屏布局：

- 屏幕上半部分：运行 `clock_demo` 内核线程，绘制一个实时旋转的模拟时钟
- 屏幕下半部分：文本控制台，包含 YSOS 标题头和可滚动的日志/Shell 输出区域

下面我们实现时钟的绘制

6.2.1. 帧同步

帧同步采用自旋等待：记录帧开始的时间戳，循环读取时间直到差值达到 1000 ms，保证每秒恰好绘制一帧，我们这里没有使用 `service.rs` 中实现的 `sys_get_time` 方法，而是在重写了一个相同的 `get_time_ms` 函数获取时间，从而避免各个模块之间的耦合

```
let start = get_time_ms();
loop {
    if get_time_ms().wrapping_sub(start) >= 1000 {
        break;
    }
    core::hint::spin_loop();
}
```

6.2.2. 粗线绘制

为了在模拟时钟中绘制清晰的指针和刻度线，我们实现了 `draw_thick_line` 函数。该函数基于经典 Bresenham 直线算法，在每个步进点绘制一个 `width * width` 的实心正方形，从而实现粗线的绘制，且渲染的线条无论方向如何都保持一致粗细，不会出现断点。

```rust
fn draw_thick_line(vga: &VgaDriver, x1: i32, y1: i32, x2: i32, y2: i32,
                   color: Color, width: i32) {
    let dx = (x2 - x1).abs();
    let dy = -(y2 - y1).abs();
    let sx = if x1 < x2 { 1 } else { -1 };
    let sy = if y1 < y2 { 1 } else { -1 };
    let mut err = dx + dy;
    let mut x = x1;
    let mut y = y1;
    let half = width / 2;

    loop {
        // 以 (x, y) 为中心绘制 width×width 的实心方块
        for wy in -half..=half {
            for wx in -half..=half {
                vga.draw_pixel((x + wx) as u32, (y + wy) as u32, color);
            }
        }
        if x == x2 && y == y2 { break; }
        let e2 = 2 * err;
        if e2 >= dy { err += dy; x += sx; }
        if e2 <= dx { err += dx; y += sy; }
    }
}
```

6.2.3. 刻度绘制

表盘上共 60 个刻度位置，每秒一个。其中整小时位置（即 `s % 5 == 0`）使用较长的亮色刻度（长 14px，颜色 RGB(200, 210, 230)，宽 2px），其余位置使用较短的暗色刻度（长 6px，颜色 RGB(50, 60, 80)，宽 1px）。

每个刻度的角度为 `s * (π / 30) - π / 2`（减去 π/2 使 0 秒位置在 12 点钟方向）。内端点位置由 `inner_r` 决定，外端点由 `outer_r` 决定，两者均基于时钟半径计算。

```rust
for s in 0..60 {
    let angle = (s as f64) * (PI / 30.0) - PI / 2.0;
    let outer_r = (radius - 8) as f64;
    let inner_r = if s % 5 == 0 {
        (radius - 22) as f64  // 整点长刻度
    } else {
        (radius - 14) as f64  // 普通短刻度
    };
    let x1 = (cx as f64 + inner_r * libm::cos(angle)) as i32;
    let y1 = (cy as f64 + inner_r * libm::sin(angle)) as i32;
    let x2 = (cx as f64 + outer_r * libm::cos(angle)) as i32;
    let y2 = (cy as f64 + outer_r * libm::sin(angle)) as i32;
    draw_thick_line(&vga, x1, y1, x2, y2, color, width);
}
```

6.2.4. 时刻数字

时刻通过 `embedded_graphics` 的 `Text` + `MonoTextStyleBuilder` + 内建 `FONT_6X10` 渲染。每个数字的位置由对应角度计算得出，位于刻度线内侧约 30px 处：

```
let mut display = VgaDisplay::new(&vga);
let num_style = MonoTextStyleBuilder::new()
    .font(&FONT_6X10)
    .text_color(Color::new(200, 210, 230))
    .build();
let nums = [12u32, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11];
for (i, &num) in nums.iter().enumerate() {
    let angle = (i as f64) * (PI / 6.0) - PI / 2.0;
    let r = (radius - 30) as f64;
    let nx = (cx as f64 + r * libm::cos(angle) - 3.0) as i32;
    let ny = (cy as f64 + r * libm::sin(angle) - 5.0) as i32;
    let _ = Text::with_text_style(
        &alloc::format!("{}", num),
        Point::new(nx, ny),
        num_style,
        TextStyleBuilder::new().baseline(Baseline::Top).build(),
    ).draw(&mut display);
}
```

这里使用 `VgaDisplay`（需要持有 VGA 锁）而非 `DirectFb`，因为时钟绘制在持锁的上下文中运行，通过标准 `get_vga()` 获取帧缓冲访问权。

6.2.5. 指针（时分秒）

三条指针的角度由累计秒数 `total_seconds` 计算：

- 秒针：

`sec_angle = (total_seconds % 60) * (π / 30) - π / 2`

- 分针：

`min_angle = ((total_seconds / 60) % 60) *(π / 30) + seconds * (π / 1800) - π / 2`（包含秒的分数偏移，使分针平滑移动）

- 时针：

`hour_angle = ((total_seconds / 3600) % 12) * (π / 6) + minutes * (π / 360) - π / 2`（包含分钟的分数偏移，使时针平滑移动）

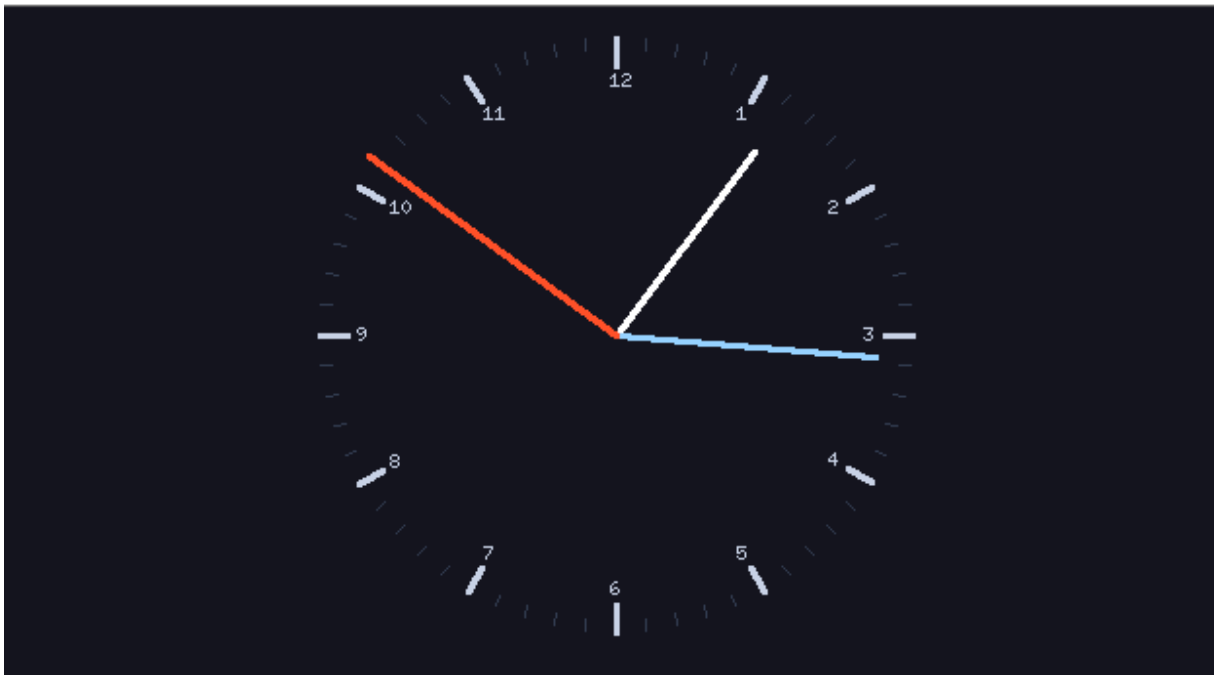
每条指针由 `draw_thick_line` 从表盘中心绘制到对应外端点，其颜色和长度以及粗度具体见下方代码，

```rust
// 时针
let hx = (cx as f64 + (radius - 42) as f64 * libm::cos(hour_angle)) as i32;
let hy = (cy as f64 + (radius - 42) as f64 * libm::sin(hour_angle)) as i32;
draw_thick_line(&vga, cx, cy, hx, hy, Color::WHITE, 3);

// 分针
let mx = (cx as f64 + (radius - 26) as f64 * libm::cos(min_angle)) as i32;
let my = (cy as f64 + (radius - 26) as f64 * libm::sin(min_angle)) as i32;
draw_thick_line(&vga, cx, cy, mx, my, Color::new(150, 210, 255), 3);

// 秒针
let sx = (cx as f64 + (radius - 4) as f64 * libm::cos(sec_angle)) as i32;
let sy = (cy as f64 + (radius - 4) as f64 * libm::sin(sec_angle)) as i32;
draw_thick_line(&vga, cx, cy, sx, sy, Color::new(255, 80, 40), 2);
```

其最终效果如下，



7. 最终布局

`VgaConsole` 支持 `y_offset` 偏移参数，使其仅占用屏幕的下半部分。在分屏模式下：

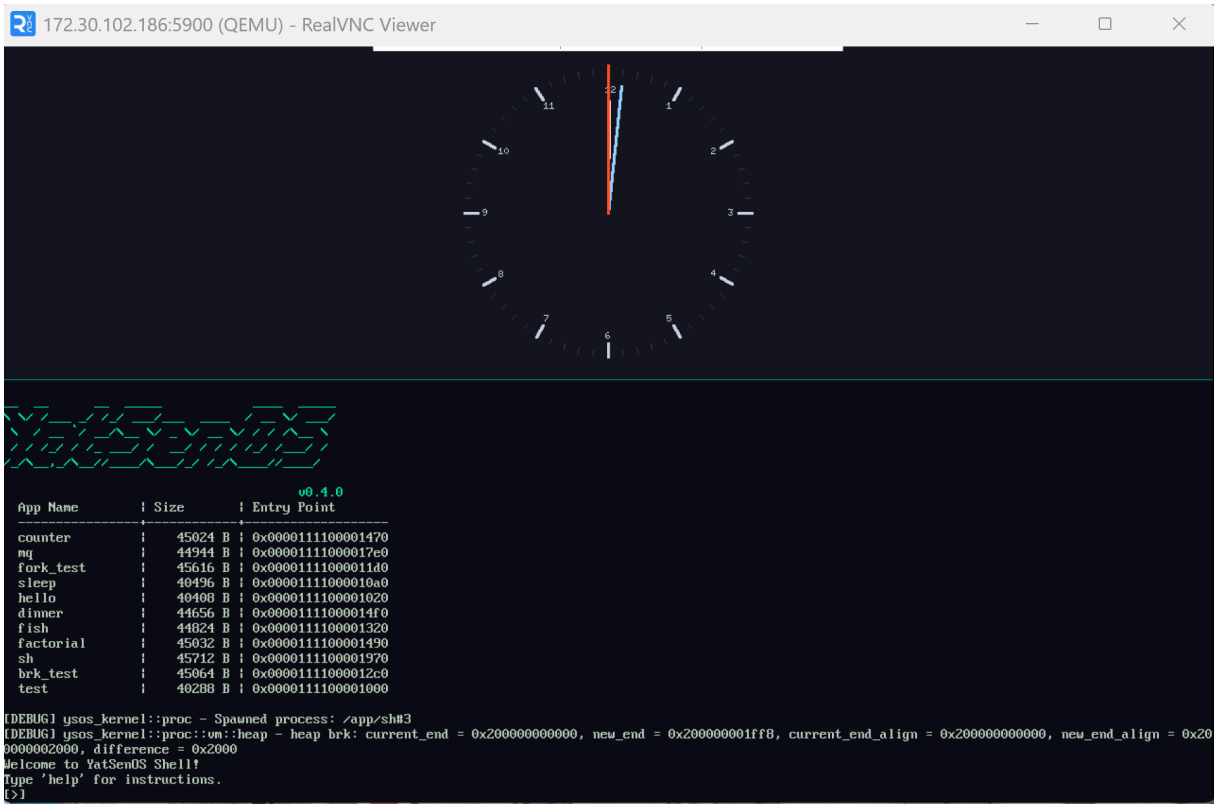
1. 屏幕上半部分用于绘制加分项中的时钟
2. 屏幕下半部分用于文本控制台

控制台进一步分为两部分：

1. Header 区：显示 YSOS ASCII 徽标，不可滚动
2. Body 区：Shell 交互 + 日志输出的可滚动区域

初始化函数 `init_vga_console_for_shell()` 在 `crates/kernel/src/drivers/vga_text.rs` 中实现，返回时钟区域的高度（像素），供 `clock_demo` 进程进行时钟绘制。

最终效果如下，使用 `make gui MODE=release DISPLAY_BACKEND=vnc` 命令进行运行，然后使用 RealVNC Viewer 连接 GUI，



从而，我们成功实现了 shell 的 VGA 显示输出！